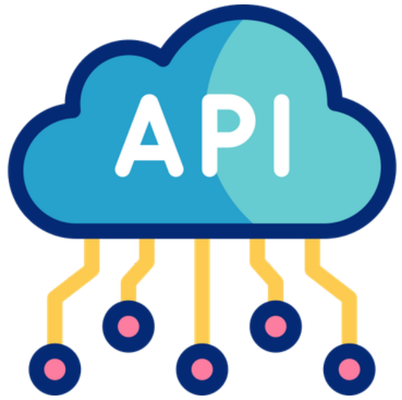
Introduction to API

Webscraping & API



Sarra Ben Yahia | José Ángel García Sánchez

What’s an API?

Definition

A set of rules that lets two applications talk to each other: one sends a request, the other sends back a response, in a format both sides agree on.

You (client)

Pick from the menu and place an order

→

The waiter (API)

Carries your request to the kitchen and brings back your dish

→

The kitchen (server)

Cooks the dish. You never see how.

> Key idea: you only need to know what to ask and what you’ll get back, not how the other app works inside.

Thousands of public APIs exist (weather, maps, payments, AI…), for example Reddit, X (formerly Twitter), and Facebook

Explore free APIs: https://github.com/public-apis/public-apis

Role of an API

01 Exchange data

Share data between different applications in a standardized format.

02 Automate

Let systems interact with no manual work, e.g. transferring information automatically.

03 Integrate

Add new services or features without rebuilding everything from scratch.

04 Control access

Decide who can use an application’s resources: public, private or partner APIs.

In short: an API is a bridge between two programs. They agree on how to talk, and the data flows safely, without anyone copying it by hand.

What are web APIs?

Definition

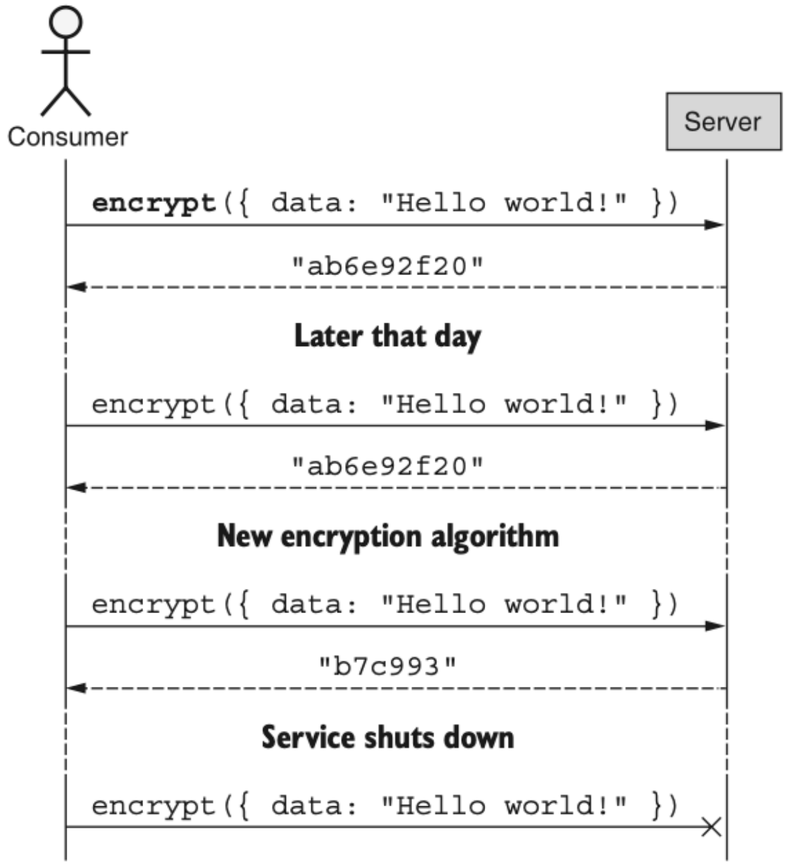
An API exposed over a network (usually the internet) and used remotely by lots of different people.

Library vs. web API

Library — You install your own copy. The builders can change their code; your version stays the same.

Web API — There is no copy. Every change the builders make reaches you, whether you asked for it or not.

> **As a consumer, you depend on choices you don’t control.**



Same call, different results: the builders switch algorithm, then shut the service down. The consumer can’t do anything about it.

Why build a web API?

What's a drawback for the consumer is the main advantage for the builder: they keep full control.

Ship changes instantly

Fix, update or retire the API once, and every user gets it right away.

Keep the secret sauce

Offer what an algorithm does without giving away its code.

Hide the heavy lifting

Run costly computation, like ML models, on your servers, not on users' laptops.

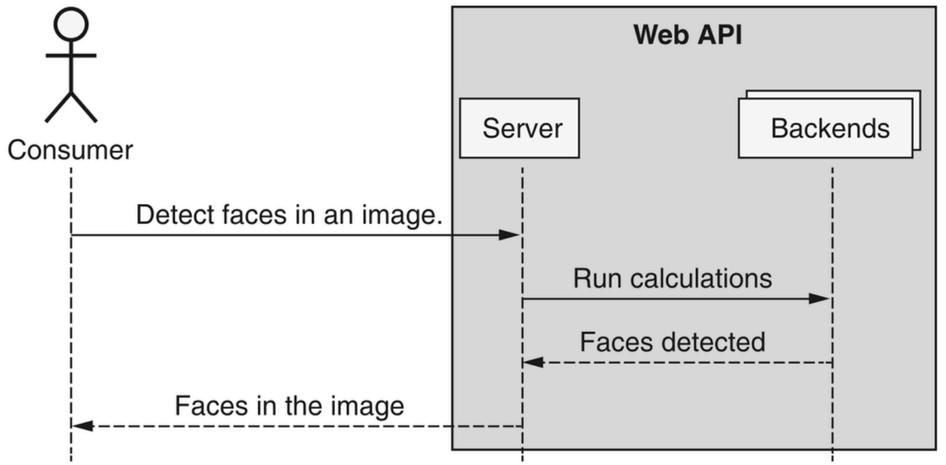
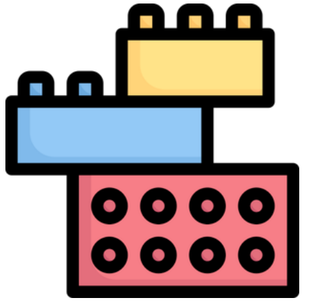


Figure 1.2 An example of a web API hiding the computational power needed

Why do APIs matter?

- Most software is built for humans: mouse, keyboard and screens.
- Computers struggle with interfaces made for people: a purely cosmetic change can break a bot. That’s why screen-scraping and RPA are so fragile.
- APIs are interfaces designed for computers: no visuals to break, and they evolve in compatible ways.
- They also enable composition: APIs snap together like Lego bricks to build something bigger than the sum of their parts.



What are resource-oriented APIs?

- Most web APIs act like servants: you ask, they do.
  - e.g. predictWeather(postalCode=10011), encrypt(data=...), sendEmail(to=...). Calling a function that runs on a far-away computer is a Remote Procedure Call (RPC). The focus is on actions.

Why aren't all APIs RPC-oriented?

- Stateless call: it works on its own, with no context. e.g. the weather for one postcode.
- Stateful call: it depends on earlier calls or stored data. e.g. forecasts for "my saved cities".
- RPC is great for stateless calls, but becomes a poor fit as soon as state comes in.

What are resource-oriented APIs?

Example: a stateful flight-booking API built with RPCs

Method	Description
`ScheduleFlight()`	Schedules a new flight
`GetFlightDetails()`	Shows information about a specific flight
`ShowAllBookings()`	Shows all travel plans currently booked
`CancelReservation()`	Cancels an existing flight reservation
`RescheduleFlight()`	Reschedules an existing flight to another date or time
`UpgradeTrip()`	Upgrades from economy class to first class

What are resource-oriented APIs?

Example: a stateful flight-booking API built with RPCs

Method	Description
`ScheduleFlight()`	Schedules a new flight
`GetFlightDetails()`	Shows information about a specific flight
`ShowAllBookings()`	Shows all travel plans currently booked
`CancelReservation()`	Cancels an existing flight reservation
`RescheduleFlight()`	Reschedules an existing flight to another date or time
`UpgradeTrip()`	Upgrades from economy class to first class

- Each method is clear, but you must memorise them all: is it a “flight” or a “reservation”?
- To see my bookings: ShowAllBookings(), ShowAllFlights() or ListFlights()? No standard, no guessing.

What are resource-oriented APIs?

The fix: standardize the resources and the methods

RPC	Description
`Create<Resource>()`	Creates a new `<Resource>`
`Get<Resource>()`	Shows information about a specific `<Resource>`
`List<Resources>()`	Shows a list of all existing `<Resources>`
`Delete<Resource>()`	Deletes an existing `<Resource>`
`Update<Resource>()`	Updates an existing `<Resource>` in place

- Resources: the “things” the API stores and manages, e.g. a flight reservation.
- Standard methods: a small, fixed set of actions that works the same on every resource.
- Every call follows one pattern: <StandardMethod><Resource>()

What are resource-oriented APIs?

One resource × five standard methods = the whole API

Method		Resource		Methods
Create				CreateFlightReservation()
Get				GetFlightReservation()
List	×	FlightReservation	=	ListFlightReservations()
Delete				DeleteFlightReservation()
Update				UpdateFlightReservation()

Is resource-oriented always better? No: RPC still fits some cases, especially stateless calls. But it's usually much easier to learn: if you know the five standard methods, one new resource name gives you five new calls for free. Patterns beat memorising a custom design.

What makes an API good?

Before judging an API, ask why it should exist at all. It usually comes down to two conditions:

- We have some functionality that some users want.
- Those users want to use it programmatically, from their own code.
- Example: an amazing translation engine could simply be a mobile app. It only deserves an API if people want to write programs that translate text.
- Once both are true, we can ask what makes the API good: operational, expressive, simple and predictable.

What are the desirable qualities of an API?

- Operational: it actually does the job (e.g. it translates text), and meets non-functional needs like speed and accuracy.
- Expressive: users can say clearly what they want. If people call TranslateText() in a loop to guess a language, the API is missing a DetectLanguage() method.
- Simple: “as simple as possible, but no simpler”. Not one ExecuteAction() that does everything: make the common case easy and the advanced case possible.
- Predictable: no surprises. The same concept has the same name everywhere: text in every method, not inputText here and content there.

Summary

- An interface is a contract: it defines how two systems talk to each other.
- An API is an interface between computer systems. It can be a library you download or a web API you call over the network.
- Web APIs expose functionality over a network and hide what's behind it: the code and the computing power.
- APIs matter because computers can't use screens reliably, and APIs combine like Lego bricks.
- Resource-oriented APIs keep things simple: a few standard methods applied to a limited set of resources.
- There's no single definition of "good", but good APIs are operational, expressive, simple and predictable.

Types of web APIs

REST is the default, but not the only option

REST

Resources + HTTP methods, usually JSON. Simple, cacheable, everywhere. Our focus.

GraphQL

One endpoint; the client asks for exactly the fields it needs. Great for complex front-ends.

SOAP

XML envelopes and strict contracts (WSDL). Still common in banking and legacy enterprise systems.

gRPC

Binary Protocol Buffers over HTTP/2. Fast service-to-service calls, common for ML model serving.

In this course we build REST APIs with FastAPI, following resource-oriented design.

REST in one slide

REpresentational State Transfer: an architectural style, not a protocol

- Client-server: the front-end and the back-end evolve independently.
- Stateless: every request carries everything needed (e.g. the token). The server keeps no session.
- Uniform interface: resources have URLs (/students/42), we act on them with HTTP methods, and exchange representations (JSON).
- Cacheable: responses say whether they can be cached (Cache-Control, ETag).
- Layered system: proxies, load balancers or gateways can sit in between without the client knowing.

Why it matters: stateless servers are easy to scale. Just add more copies behind a load balancer.

Anatomy of an HTTP exchange

Every call is one request and one response, both plain text

Request (client → server)

```
POST /students HTTP/1.1
Host: api.mosef.fr
Authorization: Bearer eyJhbGci...
Content-Type: application/json

{"name": "Ana", "year": 2}
```

Response (server → client)

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /students/42

{"id": 42, "name": "Ana",
 "year": 2}
```

- Request = method + path + headers + optional body.
- Response = status code + headers + optional body.
- Headers carry metadata: auth, content type, caching, where the new resource lives.

FastAPI uses HTTP request methods

HTTP 		CRUD 
GET	→	Read: retrieve data
POST	→	Create a new resource
PUT	→	Replace the entire resource
PATCH	→	Update only the fields sent
DELETE	→	Delete the resource

Other HTTP methods

HTTP 		Purpose 
HEAD	→	Like GET, but headers only
OPTIONS	→	Allowed methods (CORS preflight)
TRACE	→	Debug loop-back, rarely used

Safe and idempotent methods

What happens if the client sends the same request twice?

Safe

GET, HEAD, OPTIONS
Only read. Never change data on the server.

Idempotent

Safe methods + PUT, DELETE
Sending it 1 or 10 times leaves the same final state.

Neither

POST (and PATCH, in general)
Sending it twice can create two students.

- Why care: networks fail. If a response is lost, the client doesn’t know whether the request worked.
- Idempotent requests can simply be retried.
- For POST, send an idempotency key (a unique request ID) so the server can detect duplicates.
- DELETE is idempotent on state, but the second call may return 404 instead of 204.

Status code 

FastAPI response status code

- An HTTP Status Code is used to help the Client (the user or system submitting data to the server) to understand what happened on the server side application.
- Status Codes are international standards on how a Client/Server should handle the result of a request.
- It allows everyone who sends a request to know if their submission was successful or not.

Status code 

FastAPI response status code

1XX	→	Informational Response
2XX	→	Success
3XX	→	Redirection
4XX	→	Client errors
5XX	→	Server errors

Status code 

2XX Successful status code

200 → OK: standard success, e.g. a GET that returns data.

201 → Created: a new resource was created, typically by POST. Return it in the body.

204 → No Content: success, with nothing to send back. Typical for DELETE.

Status code 

4XX Client errors status code

400	→	Bad Request: malformed request or a broken business rule.
401	→	Unauthorized: not authenticated (missing or invalid token).
404	→	Not Found: the requested resource doesn’t exist.
422	→	Unprocessable Entity: FastAPI’s answer when Pydantic validation fails.

Status code 

5XX Server status code

500: Internal Server Error → Generic error: something unexpected went wrong on the server.

Also common: 502 Bad Gateway and 504 Gateway Timeout (a proxy or upstream service failed), 503 Service Unavailable (overloaded or down for maintenance). In FastAPI, a 500 is usually an unhandled Python exception.

More 4XX codes you will meet

Pick the most precise code: the client can react to it automatically

403 Forbidden

We know who you are, but you are not allowed. E.g. a student calling an admin-only route.

405 Method Not Allowed

The URL exists but not with this method. E.g. DELETE on a read-only route.

409 Conflict

The request clashes with the current state. E.g. creating a user whose email already exists.

429 Too Many Requests

Rate limit reached. The Retry-After header says how long to wait.

401 vs 403: 401 = “who are you?” (missing or invalid token). 403 = “I know you, and the answer is no.”

Retrying failed requests

Some errors are temporary. Retry them, but politely

- Retry: 429, 502, 503, 504 and network timeouts.
- Don’t retry other 4XX: the request itself is wrong, sending it again won’t help.
- Wait longer each time (exponential backoff: 1s, 2s, 4s…) and add random jitter so clients don’t all retry together.
- If the server sends Retry-After, obey it.
- Only retry POST if it carries a request ID the server can deduplicate.

```
import random, time, requests

RETRYABLE = {429, 502, 503, 504}

def get_with_retry(url, max_tries=5):
    for attempt in range(max_tries):
        r = requests.get(url, timeout=5)
        if r.status_code not in RETRYABLE:
            return r
        wait = r.headers.get("Retry-After")
        delay = float(wait) if wait else 2 ** attempt
        time.sleep(delay + random.uniform(0, 1))
    r.raise_for_status()
```

RESTful API Design

Resources

The data elements your application manages

IDs

Unique resource identifiers

URLs

Structured resource and ID strings

Verbs or actions

Terms that accompany URLs for different purposes

Examples

verbs / resources

verbs / resources / id

Designing resources

Nouns in the URL, verbs in the HTTP method

- Plural nouns for collections: /students, /models.
- Nested only for real ownership: /students/42/grades.
- IDs are opaque strings: clients must not guess or build them.
- An action that doesn’t fit CRUD? Use a custom method: POST + “:verb”.
- Long jobs (model training) return an operation to poll, not a 30-minute request.
- Be consistent: same names, same casing, same pagination everywhere.

```
# Good
GET    /students
GET    /students/42
POST   /students
PATCH  /students/42
DELETE /students/42
POST   /models/7:predict
POST   /models/7:train   → 202 + /operations/abc

# Avoid
GET    /getAllStudents
POST   /students/42/delete
GET    /student_list?action=create
```

Large collections

Never return 1 million rows in one response

- Pagination: return one page plus a way to get the next one.
- Offset (?skip=40&limit=20) is simple but slow and unstable when data changes.
- Token (?page_token=…) is opaque and stable: the server decides what “next” means.
- Filtering: ?year=2&domain=apprenticeship, validated like any input.
- Partial update: PATCH only the fields sent; field masks say which ones.

```
@app.get("/students")
def list_students(
    year: int | None = None,
    page_size: int = Query(20, le=100),
    page_token: str | None = None,
):
    rows, next_token = repo.page(
        year, page_size, page_token)
    return {"results": rows,
            "next_page_token": next_token}

# Last page: next_page_token is null
```

Reliability and versioning

Your API is a contract: other people’s code depends on it

Idempotency keys

- The client sends a unique ID with each POST:
- Idempotency-Key: 7f3c…
- The server stores the key with the response.
- Same key again → return the saved response, don’t redo the work.
- Same key, different body → 409 Conflict.

Versioning

- Put the major version in the path: /v1/students.
- Compatible: adding an optional field or a new route.
- Breaking: renaming or removing a field, changing a type or a default.
- Breaking change → /v2, keep /v1 running and announce a deprecation date.

API in Python

Several Python frameworks to build API



FastAPI



Flask



Django

What is FastAPI

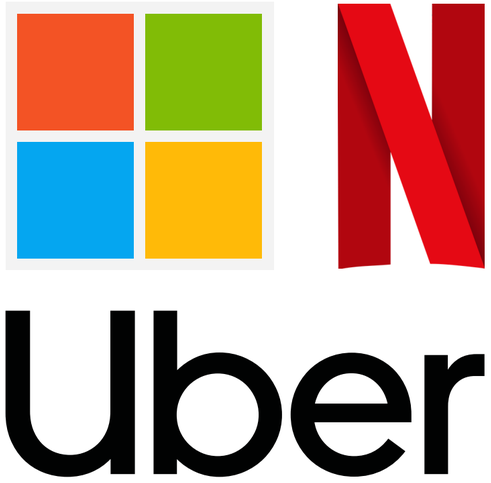
FastAPI Python web-framework

Fast to develop with and fast to run: as fast as Node.js and Go in some cases (I/O-bound workloads).
The key characteristics are:

- Few bugs
- Quick
- Easy
- Robust
- Standard
- RESTful

Official documentation

FastAPI users



Starlette

- FastAPI’s web layer is built on Starlette
- Starlette can run standalone or as a library for other frameworks.
- Handles core web tasks: HTTP request parsing & response generation (similar role to Werkzeug in Flask).
- ASGI-native (modern async standard) vs traditional WSGI used by Flask/Django.
- Async I/O reduces blocking on DB/file/network calls → better concurrency & efficiency.
- Among the fastest Python web stacks, approaching Go and Node.js performance.



How the labs work

Hands-on exercises in the API-Course repo, folder lab/

- 10 labs: each folder has an exercise.py full of TODOs and a test file that grades it.
- A lab is done when all its tests are green.
- A “Lab” slide in the deck tells you when to stop and code.
- Labs 00–03 are the core; 04–09 go further. 07 and 08 are the building blocks of the final project.
- A TODO ending with a question? Answer it in a comment: it may come up in the oral exam.

```
# once, at the root of the repo
uv sync

# grade one lab
uv run pytest lab/lab01_first_endpoints

# all labs, stop at the first failure
uv run pytest lab -x

# only the tests whose name contains "patch"
uv run pytest lab/lab03_resource_oriented -k patch

# run a lab's API, then open /docs
uv run uvicorn lab.lab01_first_endpoints.exercise:app --reload
```

API Endpoints

FastAPI Python web-framework

An API endpoint is a specific URL or URI (Uniform Resource Identifier) that serves as the destination for making a request to an API (Application Programming Interface). It acts as the "access point" for interacting with a specific resource or service that the API provides.

```python
@app.get("/annuaire")
async def annuaire():
    class_list = ["Lea", "Axel"]
    return {"annuaire": class_list}
```

Endpoint example: https://mosef.com/annuaire → GET method

API Parameters

FastAPI Python web-framework

There are different ways to pass information to an API:

- **Path**: Adding parameters directly in the URL path (e.g., /api/users/{id}).
- **Query**: Including parameters in the URL after a question mark (e.g., /api/users?id=123).
- **Header**: Sending information as part of the HTTP request header (e.g., Authorization: Bearer <token>).
- **Body**: Including data in the body of the request, typically for POST, PUT, or PATCH requests, in formats like JSON or XML."

API Parameters: Path

FastAPI Python web-framework

Path parameters are request parameters that have been attached to the URL. Usually defined as a way to find info **based on location**. Think of a computer file system, you can identify resources based on the file path:

/Users/Mosef/Documents/Github/API_course/lesson_1
https://mosef.com/annuaire/2023

```
@app.get("/annuaire/{year}")
async def annuaire_year(year: int):
    selected_year = annuaire[annuaire["year"]==year]
    return {"annuaire": selected_year.to_dict("records")}
```

API Parameters: Query

FastAPI Python web-framework

Query parameters are **key-value pairs** appended to the URL after a “**?**”. They are used to pass additional data to the server in a structured format, following the pattern **name=value**, and are typically separated by & for multiple parameters.

https://mosef.com/annuaire?apprenticeship=health

```
@app.get("/annuaire")
async def annuaire(apprenticeship: str):
    selected = annuaire[annuaire["domain"]==apprenticeship]
    return {"annuaire": selected.to_dict("records")}
```

API Parameters: Header

FastAPI Python web-framework

Headers are **key-value** pairs sent as part of the HTTP request, **separate from the URL and body.** They provide additional context or metadata about the request, such as **authentication tokens**, **content type, or caching details**. For example, the headers might include the format the client expects in the response or information necessary for the server to process the request correctly. In FastAPI, read one with Header() or get the token through a security dependency.

headers = {"Authorization": "Bearer <token>", "Content-Type": "application/json"}

API Parameters: Body

FastAPI Python web-framework

The **body** is the section of an HTTP request or response that contains the actual data being transmitted. Unlike headers or query parameters, the body is not visible in the URL and can carry more complex data, such as JSON, XML, or form data. This is where you send or receive the main information, especially when making **POST, PUT,** or **PATCH** requests, where data needs to be created or updated on the server.

API Parameters: Body

FastAPI Python web-framework

```
class StudentRecord(BaseModel):
    id: int
    last_validate_level: str = Field(min_length=2, max_length=2)
    last_average_mark: int
    dedicated_hours: int

@app.post("/predict_passing_year")
def predict(student_info: StudentRecord):
    df_record = pd.DataFrame([student_info.model_dump()])
    pred = model.predict(df_record.drop(columns="id"))
    return {"score": float(pred[0])}
```

Pydantic

FastAPI Python web-framework

Pydantic (v2) is the library FastAPI uses to define data models with type hints. It parses incoming JSON, converts compatible types ("23" → 23) and checks constraints. If validation fails, FastAPI returns a 422 with the details, automatically.

Constraints go in Field(): min_length / max_length for strings, ge / le for numbers. model_dump() turns a model back into a dict.

```
class StudentRecord(BaseModel):
    id: int
    last_validate_level: str = Field(min_length=2, max_length=2)
    last_average_mark: float = Field(ge=0, le=20)
    dedicated_hours: int = Field(ge=0)
```

OpenAPI

OpenAPI document defines

- Schema
- Data Format
- Data Type
- Path
- Object
- Others



Note: FastAPI generates the OpenAPI schema automatically
helping to be RESTful compliant so individuals can use easily the API

OpenAPI


```
{
  "openapi": "3.1.0",
  "info": {
    "title": "FastAPI",
    "version": "0.1.0"
  },
  "paths": {
    "/": {
      "get": {
        "summary": "Root",
        "operationId": "root__get",
        "responses": {
          "200": {
            "description": "Successful Response",
            "content": {
              "application/json": {
                "schema": {

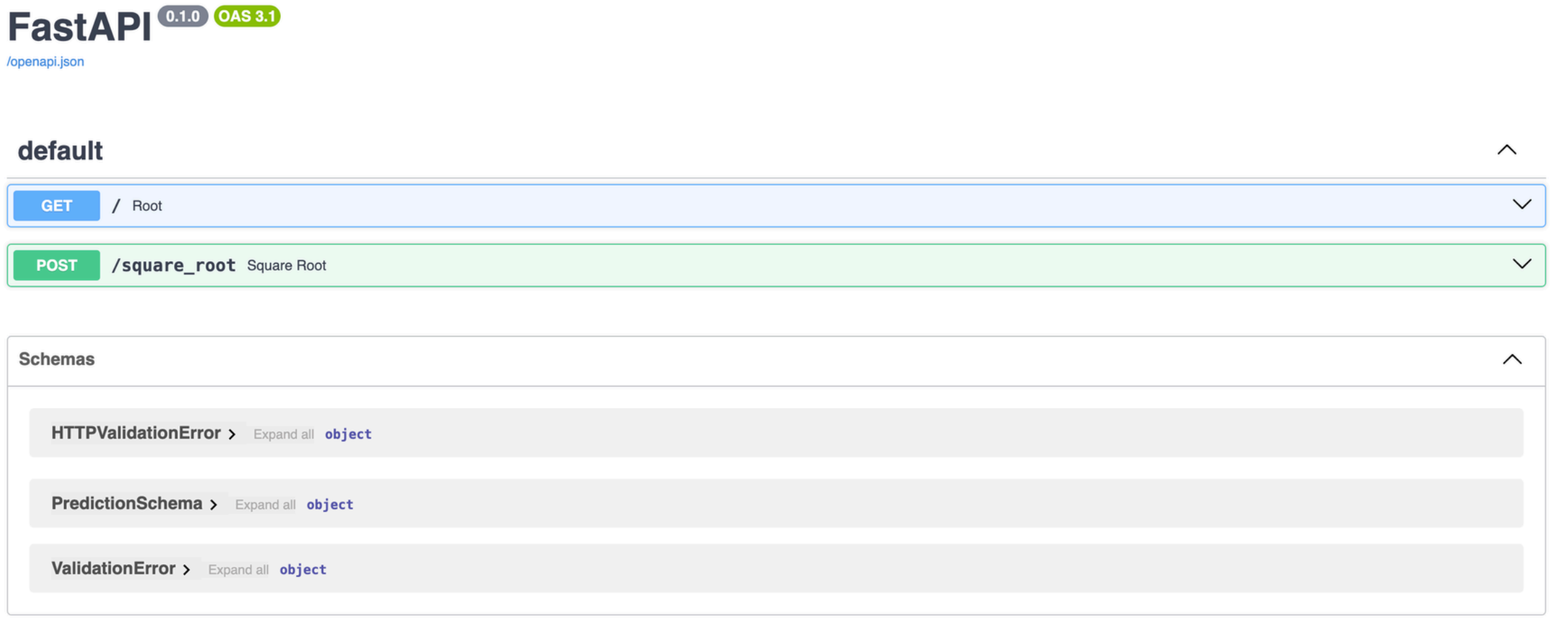
                }
              }
            }
          }
        }
      }
    },
```

```
"/square_root": {
  "post": {
    "summary": "Square Root",
    "operationId": "square_root_square_root_post",
    "parameters": [
      {
        "name": "input_data",
        "in": "query",
        "required": true,
        "schema": {
          "type": "integer",
          "title": "Input Data"
        }
      }
    ],
```




The OpenAPI schema is accessible from http://127.0.0.1:8000/openapi.json

Swagger UI



The swagger is accessible from http://127.0.0.1:8000/docs

Lab 01 · Your first endpoints

Time to code: lab/lab01_first_endpoints

- Build the “MoSEF Annuaire” API: GET /hi, /health and /annuaire.
- A path parameter: GET /annuaire/{year}.
- A header: GET /whoami reads X-Client-Name.
- A JSON body: POST /annuaire, with the right status code.
- Check the title and routes in /docs and /openapi.json.
- Recap: slides “API Endpoints” to “Swagger UI” · FastAPI book ch. 3

```
# run the API and play with it
uv run uvicorn \
  lab.lab01_first_endpoints.exercise:app --reload

# open in the browser
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/openapi.json

# grade your work
uv run pytest lab/lab01_first_endpoints
```

Lab 02 · Pydantic models

Time to code: lab/lab02_pydantic_models

- Field() constraints: positive id, 2-letter level, mark between 0 and 20.
- A field validator, and a model validator across two fields (booking end after start).
- Clean a messy list of rows, data-science style.
- Input vs output models: POST /users never sends the password back.
- Recap: slides “Body” and “Pydantic” · repo data_models/ · FastAPI book ch. 5

```
class UserIn(BaseModel):
    username: str
    password: str    # TODO: at least 8 chars

class UserOut(BaseModel):
    username: str    # no password here

@app.post("/users", response_model=UserOut,
          status_code=201)
def create_user(user: UserIn): ...

# grade your work
uv run pytest lab/lab02_pydantic_models
```

Lab 03 · From RPC to resources

Time to code: lab/lab03_resource_oriented

- Map each RPC call of the flight-booking example (ScheduleFlight, ShowAllBookings…) to a standard method.
- Build one resource, /flightReservations, with the five standard methods.
- Right verb and status code for each: 201, 200, 204, 404.
- Generate opaque ids in Crockford Base32.
- Recap: slides “What are resource-oriented APIs?”, “HTTP methods”, “Status code” · API Design Patterns ch. 1, 3, 6, 7

```
# the whole API = 1 resource x 5 methods
POST    /flightReservations
GET     /flightReservations/{id}
GET     /flightReservations
PATCH   /flightReservations/{id}
DELETE  /flightReservations/{id}

# grade your work
uv run pytest lab/lab03_resource_oriented

# only the PATCH tests
uv run pytest lab/lab03_resource_oriented -k patch
```

Calling APIs from Python

The client side: how your notebook or app consumes an API

- requests: the classic, simple and synchronous.
- httpx: same API, plus async support (AsyncClient).
- Always set a timeout: the default is to wait forever.
- raise_for_status() turns 4XX/5XX into exceptions.
- To explore: /docs (Swagger UI), curl, Postman or Bruno.

```
import requests

r = requests.post(
    "http://localhost:8000/predict",
    json={"age": 23, "year": 2},
    headers={"Authorization": f"Bearer {token}"},
    timeout=10,
)
r.raise_for_status()
print(r.status_code, r.json())

# async version
async with httpx.AsyncClient() as client:
    r = await client.get(url, timeout=10)
```

Layered architecture

Don’t put everything in main.py: split by responsibility

- Web: routes, HTTP status codes, request/response models. Knows nothing about SQL.
- Service: business rules (can this student enroll? which model version?).
- Data: reads and writes the database. Knows nothing about HTTP.
- Model: Pydantic schemas shared between layers.
- Each layer only calls the one below, so you can test and replace it on its own.

```
app/
├── main.py          # creates FastAPI(), includes routers
├── web/
│   └── students.py  # @router.get(...)
├── service/
│   └── students.py  # business logic
├── data/
│   ├── database.py  # engine, session
│   └── students.py  # SQL queries
├── model/
│   └── student.py   # Pydantic schemas
└── tests/
```

Dependency injection

Depends(): FastAPI gives each route what it needs

- A dependency is a function FastAPI calls before your route and passes in the result.
- Typical uses: database session, current user, settings, pagination parameters.
- With yield, code after it runs once the response is sent (close the session).
- Scope: one route, a whole router, or the entire app.
- In tests, swap any dependency with app.dependency_overrides.

```
from fastapi import Depends

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/students/{student_id}")
def read_student(student_id: int,
                 db: Session = Depends(get_db)):
    return service.get_student(db, student_id)
```

Error handling

Turn Python errors into clear HTTP answers

- Raise HTTPException(status_code, detail) in the web layer.
- The service raises domain errors (Missing, Duplicate), not HTTP ones.
- Map them once: Missing → 404, Duplicate → 409.
- Keep one error format everywhere so clients can parse it.
- Never leak stack traces or SQL in a 500: log them, return a generic message.

```
class Missing(Exception): ...
class Duplicate(Exception): ...

@app.exception_handler(Missing)
def missing_handler(request, exc):
    return JSONResponse(status_code=404,
        content={"detail": str(exc)})

@app.get("/students/{student_id}")
def read_student(student_id: int):
    return service.get_student(student_id)
    # raises Missing → 404 automatically
```

Persistence

Store data in a real database, not in a Python list

- SQLAlchemy ORM: Python classes mapped to tables.
- A session = one unit of work. Open it per request with Depends(get_db).
- Two kinds of models: ORM (the table) and Pydantic (the API contract). Don’t mix them.
- SQLite to learn, PostgreSQL in production: only the URL changes.
- Alembic versions the schema: every table change is a migration.

```
class StudentORM(Base):
    __tablename__ = "students"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    year: Mapped[int]

class StudentOut(BaseModel):
    id: int
    name: str
    model_config = ConfigDict(from_attributes=True)

$ alembic revision --autogenerate -m "add year"
$ alembic upgrade head
```

Lab 04 · Layers and dependencies

Time to code: lab/lab04_layers_and_dependencies

- A creature API split into web → service → data layers, on SQLite.
- Data: parameterised SQL queries (and spot the SQL injection).
- Service: business rules, raises Missing / Duplicate.
- Web: routes, Depends(get_db), a reusable paging dependency, 404 / 409.
- Tests swap the database with dependency_overrides.
- Recap: slides “Layered architecture” to “Persistence” · FastAPI book ch. 6, 8–10, 12

```
lab04_layers_and_dependencies/
└── exercise/
    ├── web.py      # routes, Depends()
    ├── service.py  # business rules
    ├── data.py     # SQL on SQLite
    ├── model.py    # Pydantic Creature
    └── errors.py   # Missing, Duplicate

# suggested order: data → service → web
uv run pytest lab/lab04_layers_and_dependencies
```

Lab 05 · Pagination and filtering

Time to code: lab/lab05_pagination_filtering

- Parse a tiny filter language and apply it to a list.
- Build opaque page tokens that hold the offset and a fingerprint of the filter.
- Enforce a default and a maximum max_page_size.
- GET /creatures?max_page_size=&page_token=&filter=
- A bad token, filter or page size → 400, not 500.
- Recap: slide “Large collections” · API Design Patterns ch. 21, 22

```
# the List response shape
{
  "results": [ ... ],
  "next_page_token": "eyJvZmZzZXQiOi..."
}
# "" means: this was the last page

# the client only passes the token along
GET /creatures?max_page_size=10
GET /creatures?max_page_size=10&page_token=...

uv run pytest lab/lab05_pagination_filtering
```

Lab 06 · Updates and custom methods

Time to code: lab/lab06_updates_custom_methods

- PATCH with a field mask (?update_mask=): change only the fields listed.
- Partial GET: ?fields=name,location.area returns just those.
- A custom method: POST /creatures/{id}:reportSighting.
- Soft delete: DELETE only flags it, :undelete restores, :expunge removes for good.
- Wrong state (e.g. deleting twice) → 412 Precondition Failed.
- Recap: slides “HTTP methods”, “Designing resources”, “Large collections” · API Design Patterns ch. 8, 9, 25

```
PATCH  /creatures/yeti?update_mask=location.area
GET    /creatures/yeti?fields=name,location.area
POST   /creatures/yeti:reportSighting

DELETE /creatures/yeti           # deleted = true
POST   /creatures/yeti:undelete  # back again
POST   /creatures/yeti:expunge   # gone, 204

GET    /creatures?include_deleted=true

uv run pytest lab/lab06_updates_custom_methods
```

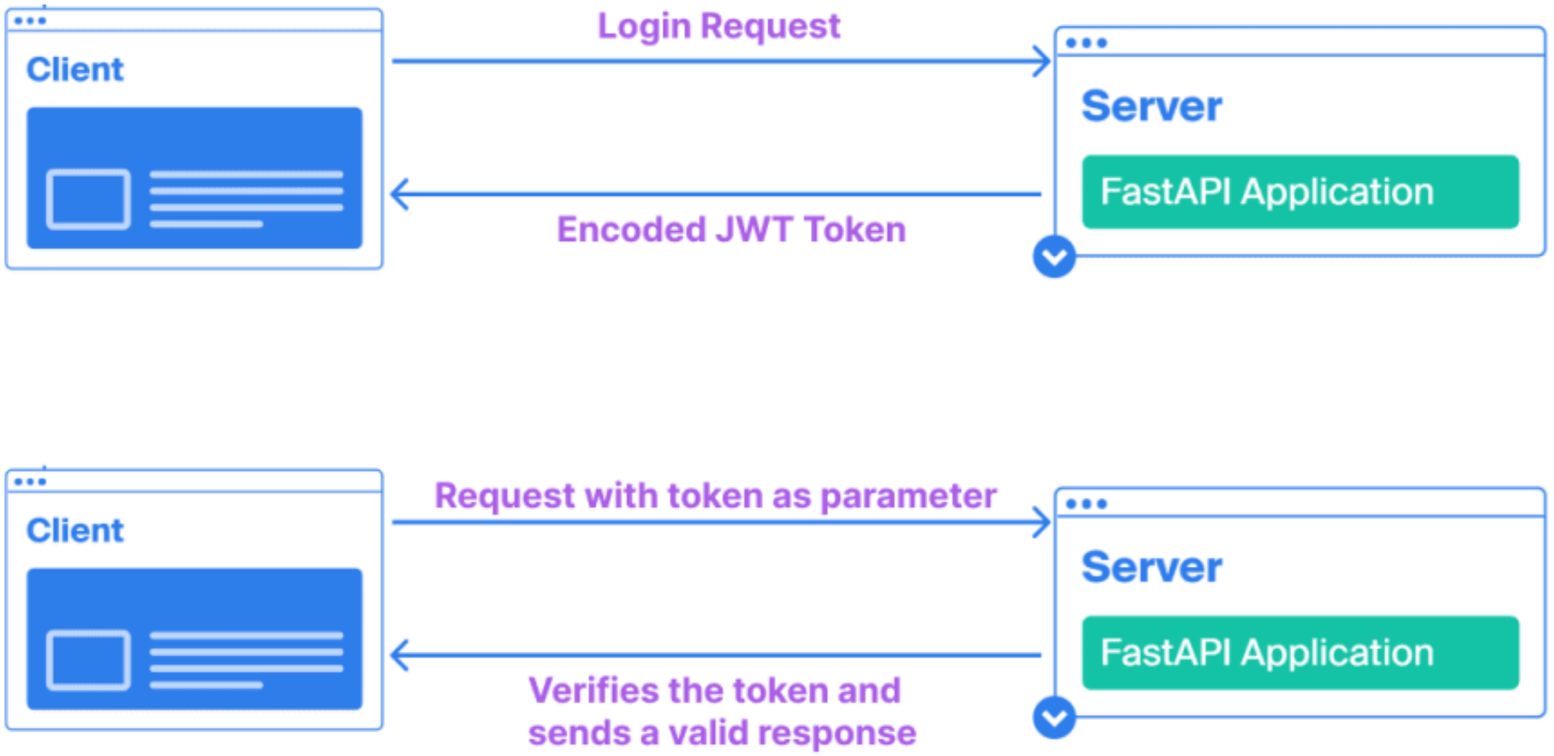
JSON Web Token (JWT)

FastAPI Python web-framework

- A JWT is a compact, self-contained token that carries JSON “claims” between two parties.
- It is signed, not encrypted: the server detects any change, but anyone can read the content.
- Typical flow: after login (authentication), the client sends it on every request; claims like role support authorization.
- Limit: it can’t be revoked before it expires, so keep exp short and use refresh tokens.

JSON Web Token (JWT)

FastAPI Python web-framework



JSON Web Token (JWT)

FastAPI Python web-framework

A JSON Web Token is created of three separate parts separated by dots (.) which include:

- Header : (a)
- Payload : (b)
- Signature : (c)

JWT example: aaaaaaaa.bbbbbbbb.cccccccc

JSON Web Token (JWT)

FastAPI Python web-framework

A JWT **header** usually consist of two parts:

- (alg) The algorithm for signing
- (typ) The type of token

HS256 = one shared secret signs and verifies. RS256 = a private key signs, a public key verifies. The header is Base64URL-encoded to form part (a).

JWT example: aaaaaaaa.bbbbbbbb.cccccccc

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

JSON Web Token (JWT)

FastAPI Python web-framework

A JWT **Payload carries the data as “claims”. Three kinds:**

- Registered: sub, exp, iat, iss, aud
- Public
- Private (your own, e.g. role)

The payload is Base64URL-encoded, not encrypted, to form part (b). Never put secrets in it.

JWT example: aaaaaaaa.bbbbbbbb.cccccccc

```
{
  "sub": "123456",
  "name": "Albert Einstein",
  "role": "admin",
  "iat": 1792600000,
  "exp": 1792601800
}
```

JSON Web Token (JWT)

FastAPI Python web-framework

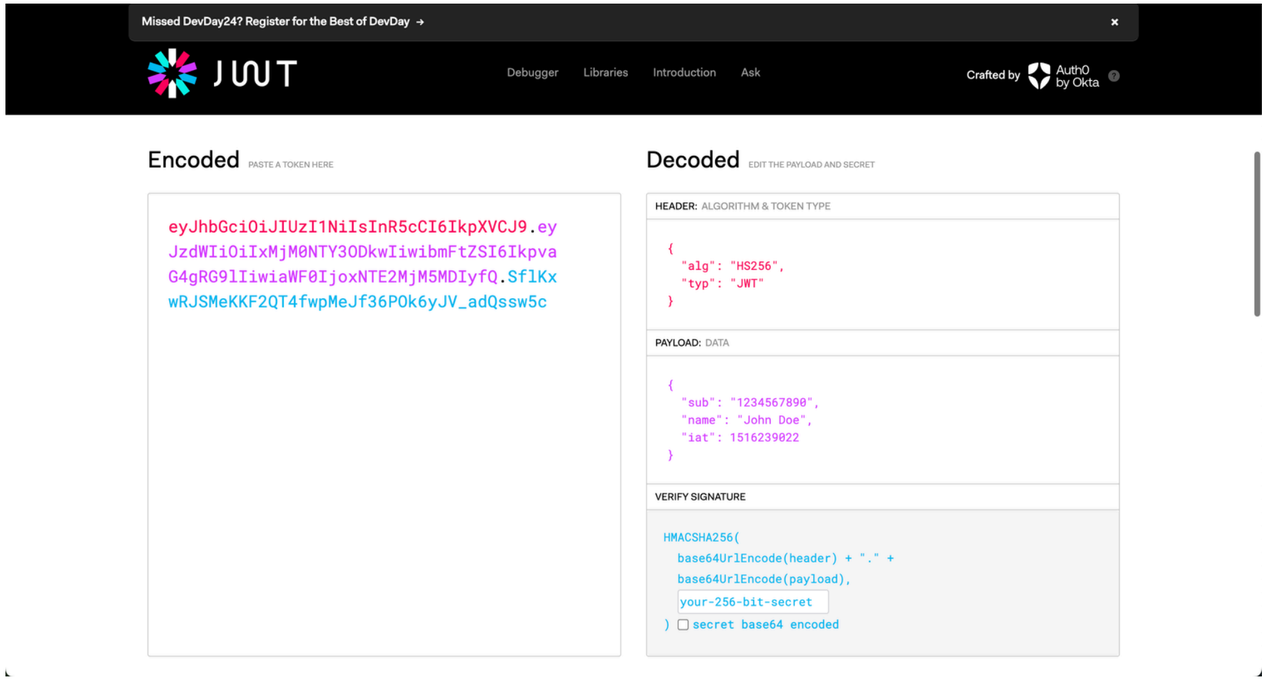
- The signature is computed from the encoded header and payload, with the algorithm from the header and a secret key.
- The secret stays on the server; the client never sees it.
- If anyone changes the header or payload, the signature no longer matches and the token is rejected (401).

JWT example: aaaaaaaa.bbbbbbbb.cccccccc

```
HMACSHA256(
  base64url(header) + "." +
  base64url(payload),
  secret
)
```

JSON Web Token (JWT)

FastAPI Python web-framework



LINK

Auth in FastAPI

From login to a protected route, in four pieces

1. Store password hashes (bcrypt/argon2), never plain passwords.
2. POST /token checks the password and returns a signed JWT.
3. OAuth2PasswordBearer reads “Authorization: Bearer …”.
4. get_current_user decodes the token: invalid or expired → 401.
5. Roles go in the token; require_admin returns 403 for others.

```
oauth2 = OAuth2PasswordBearer(tokenUrl="token")

def get_current_user(token: str = Depends(oauth2)):
    try:
        payload = jwt.decode(token, SECRET,
                             algorithms=["HS256"])
    except jwt.InvalidTokenError:
        raise HTTPException(401, "Invalid token",
            headers={"WWW-Authenticate": "Bearer"})
    return payload

def require_admin(user=Depends(get_current_user)):
    if user.get("role") != "admin":
        raise HTTPException(403, "Admins only")
    return user
```

Lab 07 · JWT authentication

Time to code: lab/lab07_jwt_auth

- Build HS256 JWTs by hand: base64url, hmac, hashlib, json. Nothing is magic.
- Verify them: bad signature or expired → InvalidToken.
- Check salted password hashes, never plain passwords.
- POST /token, a current_user dependency, and admin_user: 401 vs 403.
- A middleware that adds a header to every response.
- Recap: slides “JSON Web Token” to “Auth in FastAPI” · FastAPI book ch. 11

```
POST /token        {"username": "lea", ...}
  → 200 {"access_token": "aaa.bbb.ccc"}
  → 401 wrong password

GET /me            Authorization: Bearer aaa.bbb.ccc
  → 401 missing, forged or expired token

GET /admin/stats   token of a non-admin
  → 403 Forbidden

uv run pytest lab/lab07_jwt_auth
```

Testing your API

If it isn’t tested, it’s broken. You just don’t know it yet

- Unit tests: the service layer, with a fake data layer.
- Integration tests: real routes with TestClient, no server needed.
- Swap the DB or the auth with app.dependency_overrides.
- Test the failures too: 401, 404, 422, not only 200.
- Go further: Schemathesis generates tests from openapi.json; Locust measures load.

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    r = client.get("/health")
    assert r.status_code == 200

def test_predict_rejects_bad_input():
    r = client.post("/predict", json={"age": -3})
    assert r.status_code == 422

$ pytest -q
```

Lab 09 · Reliability

Time to code: lab/lab09_reliability

- The scenario: the response to POST /payments is lost, the client retries… and Lea gets paid twice.
- Server: deduplicate with a request_id + a SHA-256 fingerprint of the body.
- Same id, different body → 409. ?validate_only=true checks without paying.
- Client: retry only what is safe, with exponential backoff, honouring Retry-After.
- Recap: slides “Safe and idempotent methods”, “Retrying failed requests”, “Reliability and versioning” · API Design Patterns ch. 26, 27, 29

```
POST /payments
{"request_id": "7f3c", "amount": 50,
 "currency": "EUR", "to": "lea"}
  → 201   first time
  → 201   same id + same body: saved response
  → 409   same id + another body

# client side
backoff_delays()      # 1s, 2s, 4s… + jitter
parse_retry_after()   # "120" or an HTTP date
call_with_retry()     # 503 → retry, 400 → stop

uv run pytest lab/lab09_reliability
```

Types of concurrency

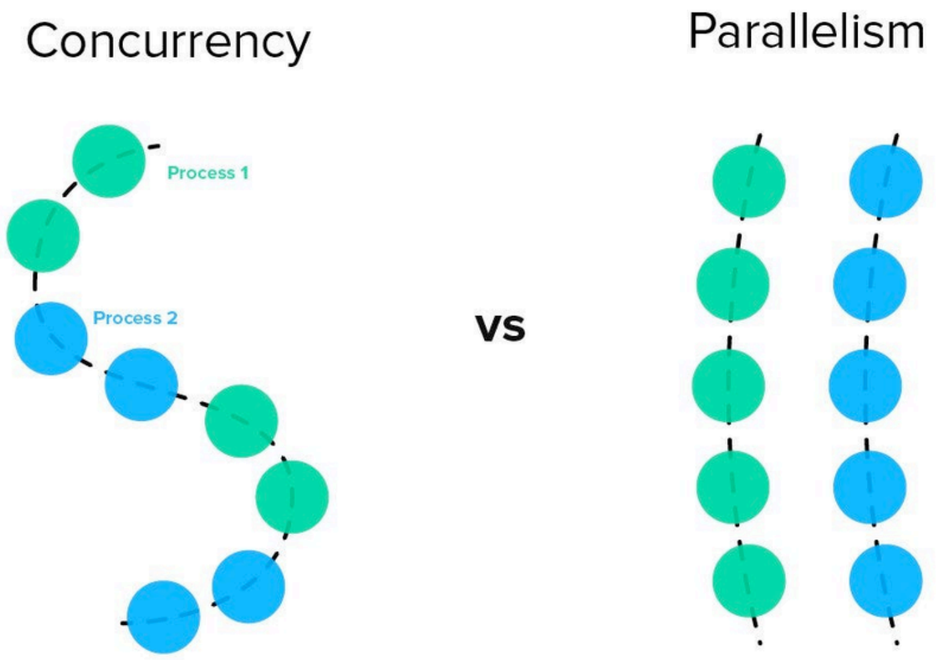
Why concurrency?

- Web apps spend time waiting on I/O (files, DB, network) — far slower than CPU
- Concurrency reduces total wait time by overlapping work
- Goal: **serve more requests** with the same hardware

Types of concurrency

Parallelism vs Concurrency

- Parallel computing: split a task across multiple CPUs at the same time (great for number-crunching)
- Concurrent computing: a CPU switches among tasks to keep busy while others wait
- Web servers are usually I/O-bound → concurrency helps most

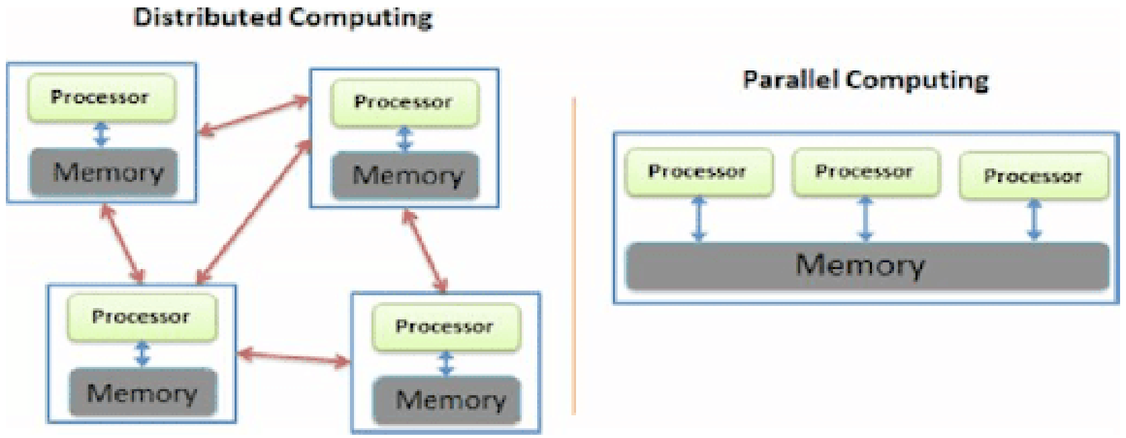


Types of concurrency

Distributed & Parallel Computing (System-Wide)

- Break big apps into pieces across cores or machines
- Powerful but complex & costly to manage (orchestrators, comms, failures)
- Our focus: small/medium apps that can run on one box, mixing sync + async

Note: Only for really big applications



Types of concurrency

OS Processes

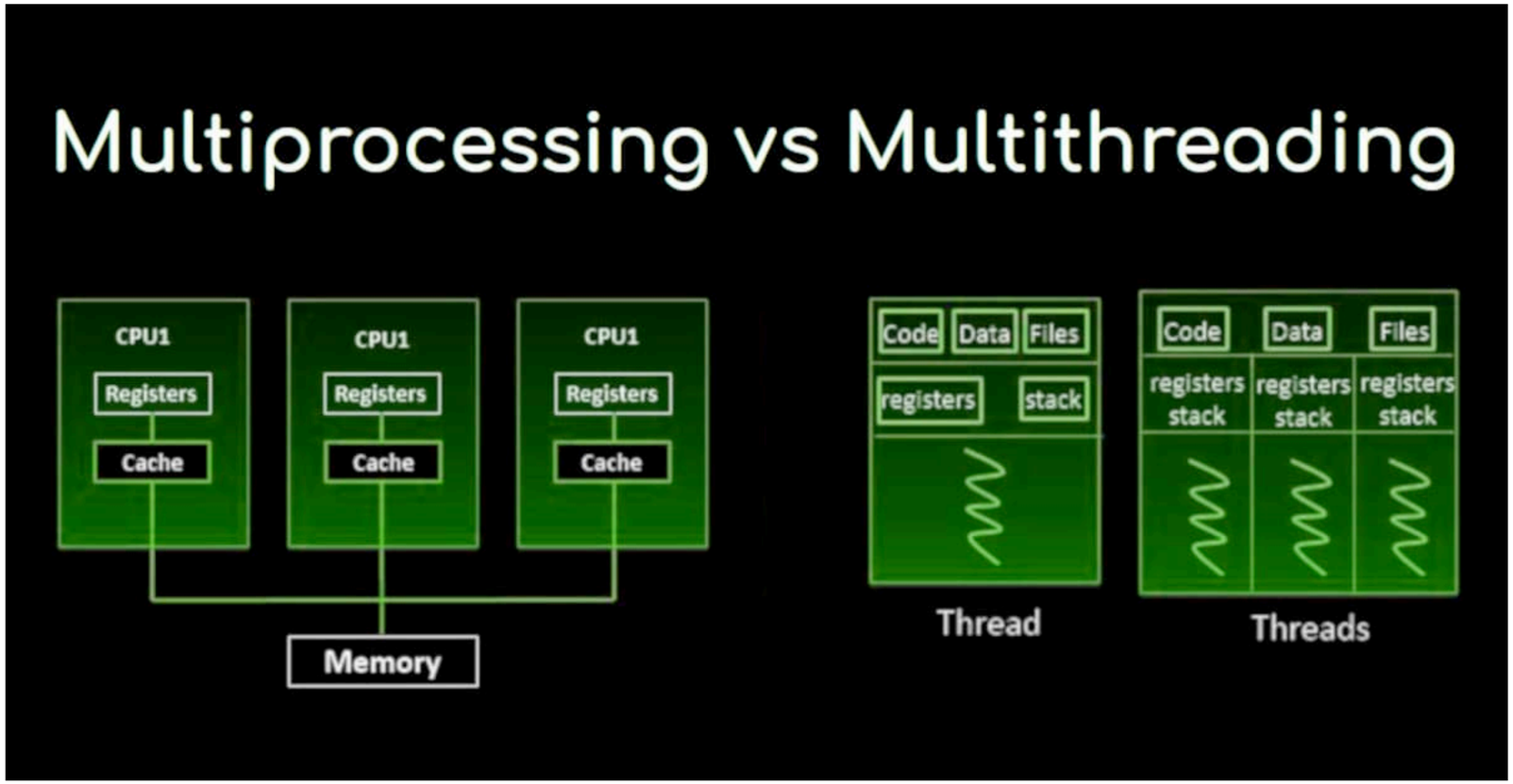
- OS schedules resources; each program runs in one or more processes
- Preemptive scheduling prevents any process from hogging the CPU
- Python for CPU-bound: use multiprocessing (multiple processes, separate memory)
- Pro: avoids GIL for CPU work
- Con: higher memory/IPC overhead

Types of concurrency

OS Threads

- Multiple threads inside one process (shared memory) via threading
- Good for I/O-bound tasks; tricky bugs (races, deadlocks)
- Higher-level concurrent.futures.ThreadPoolExecutor simplifies usage
- In FastAPI: normal def endpoints can run in threadpools automatically

Types of concurrency



Types of concurrency

Async I/O (Python async/await)

- Cooperative concurrency: tasks yield control while awaiting I/O
- Minimal overhead vs threads; great fit for many slow I/O ops
- Python: asyncio; Web standard: ASGI
- Starlette/FastAPI are ASGI-native → efficient, scalable I/O handling

Types of concurrency

Conclusion

- CPU-bound work → processes (multiprocessing, job queues)
- I/O-bound, many sockets/files/DB calls → async/await (ASGI/FastAPI)
- Simple blocking I/O in otherwise async app → threadpool offloading
- Start async for web endpoints; isolate heavy CPU tasks to workers

FastAPI / Starlette

- Built on ASGI → excellent concurrency for web workloads
- Mix and match: async endpoints + threadpools for sync code
- For heavy compute, scale with processes or external workers
- Result: high throughput and responsiveness with manageable complexity

Concurrency in FastAPI: in practice

Don’t block the event loop

- async def + a blocking call (requests.get, time.sleep, a heavy model.predict) freezes every other request.
- Plain def for blocking code: FastAPI runs it in a threadpool for you.
- async def only with awaitable libraries: httpx.AsyncClient, async DB drivers.

Rule of thumb

- I/O you can await → async def
- Blocking I/O → def (threadpool)
- CPU-heavy work (ML inference, training) → process pool or a separate worker (Celery, RQ)
- More traffic → several Uvicorn workers, roughly one per CPU core

Lab 00 · Python warm-up

Time to code: lab/lab00_python_warmup

- Type hints, and an error instead of a division by zero.
- Generators: countdown() and chunks() yield one value at a time.
- async/await: run several fake downloads concurrently with asyncio.
- For 5 real workloads, choose async, threadpool or processes.
- A good Python refresher: you can also do it at the start of the course.
- Recap: slides “Types of concurrency” · repo api/generators.py, api/async_await_asyncio.py · FastAPI book ch. 2, 4

```
async def fake_download(name, delay):
    await asyncio.sleep(delay)   # not time.sleep!
    return name

# 3 downloads of 1s each:
#   one after the other  → ~3 s
#   with asyncio.gather  → ~1 s

# async, threadpool or processes?
"train a random forest on 10M rows": "TODO"

uv run pytest lab/lab00_python_warmup
```

Lab 08 · A model and an external API

Time to code: lab/lab08_external_api_and_model

- A tiny deterministic ML model: linear regression with fixed coefficients.
- POST /predict and POST /predict:batch (1 to 100 students).
- GET /health that also returns the model version.
- GET /countries/{code}: an async call to a real API with httpx.
- Upstream errors → 404, 502 or 504. Tests swap the HTTP client for a fake.
- Recap: slides “Body”, “Calling APIs from Python”, “Concurrency in practice” · FastAPI book ch. 4, 12, 14

```
# run against the real countries API
uv run uvicorn \
  lab.lab08_external_api_and_model.exercise:app \
  --reload
# http://127.0.0.1:8000/countries/fr

# the upstream API fails? say so precisely
#   unknown country    → 404
#   upstream error     → 502 Bad Gateway
#   upstream too slow  → 504 Gateway Timeout

uv run pytest lab/lab08_external_api_and_model
```

Going to production

uvicorn --reload is for your laptop, not for real users

- Several worker processes: uvicorn --workers 4, or Gunicorn with UvicornWorker.
- HTTPS always, usually ended by a reverse proxy (Nginx, Traefik) or the cloud load balancer.
- Package it in Docker so it runs the same everywhere.
- Secrets in environment variables, never in the code or the image.
- Add a /health route, structured logs, CORS rules and rate limits.

```
# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ app/
EXPOSE 8000
CMD ["uvicorn", "app.main:app",
     "--host", "0.0.0.0", "--port", "8000",
     "--workers", "4"]

$ docker build -t mosef-api .
$ docker run -p 8000:8000 --env-file .env mosef-api
```